

DEVOIR #1 :

SYSTÈME DE GESTION DE CARGAISON DE CAMIONS D'UN ENTREPÔT

OBJECTIFS

- Analyser la complexité temporelle d'un algorithme avec deux méthodes : asymptotique et empirique.
- Résoudre un problème simple.
- Appliquer de structures de données simples (Tableaux, Listes, Piles, Files).

PROBLÉMATIQUE

Vous devez écrire un programme en Java nommé `Tp1` qui gère un processus de cargaison des camions d'un entrepôt. Un grand entrepôt possède plusieurs bâtiments d'entreposage. Le programme reçoit en entrée le nombre total de boîtes à transporter, la capacité maximale d'un camion, les positions des bâtiments d'entreposage impliqués dans la cargaison courante ainsi que le nombre de boîtes disponibles à chaque point de cargaison. Pour commencer, il faut rechercher un point de service possédant le plus grand nombre de boîtes. Les coordonnées de ce bâtiment deviennent la position courante du camion. Pour chaque cargaison, le programme doit afficher les positions des bâtiments d'entreposage situés à la distance la plus proche de la position du camion, et le nombre de boîtes restantes aux points de service. Si les distances sont identiques, vous devez prendre celle avec la coordonnée de bâtiment d'entreposage en latitude plus petite et finalement, si les latitudes sont pareilles, on prendra selon la plus petite longitude. On suppose qu'à partir de chaque édifice, vers le camion, le transport de toutes les boîtes disponibles est possible. Le nombre de boîtes à charger dans le camion ne doit pas dépasser la capacité maximale de celui-ci.

Les distances doivent être calculées en utilisant la formule de haversine (https://fr.wikipedia.org/wiki/Formule_de_haversine) avec le rayon de terre 6371000 mètres.

Hypothèses simplificatrices et directives.

Le camion reste toujours à sa position spécifiée, ce sont plutôt les monte-charges qui se déplacent pour aller chercher et charger les boîtes. Un monte-charge peut livrer toutes les boîtes disponibles à la fois. On maximise le chargement d'un camion seulement au début en plaçant le camion à la position du point de service possédant le maximum des boîtes. Ensuite, on cherche à minimiser la distance parcourue par les monte-charges.

Vous pouvez utiliser l'implémentation standard de la structure de données **Array** fournie par le Java Collections Framework. Pour résoudre ce problème, un algorithme de tri est requis. Vous devez implémenter deux algorithmes de tri distincts : l'un simple (par exemple, tri à bulles, tri par insertion) et l'autre efficace (par exemple, tri fusion, tri rapide). Ces deux algorithmes donneront lieu à deux versions distinctes de votre solution. La troisième version doit utiliser la méthode de tri native fournie par la bibliothèque Java (par exemple, **Arrays.sort()** ou **Collections.sort()**). Au total, vous devez produire trois solutions au même problème, ne différant que par la stratégie de tri employée pour ordonner les distances calculées.

Exemple :

Supposons qu'un camion doit transporter 15 boîtes de marchandise... La capacité du camion est mesurée en termes de boîtes et pour cet exemple, elle sera donc aussi de 15. Dans la liste des bâtiments de cargaison, il y a 4 points de services localisés aux coordonnées spécifiées avec le nombre de boîtes disponibles. Les données initiales et les résultats trouvés sont affichés plus bas. Pour éviter les problèmes d'encodage des caractères accentués et faciliter la procédure de correction automatique, tous les messages d'entrées et de sorties doivent être écrits en anglais et respecter à la lettre le format spécifié.

```
15 15
2 (45.515399, -73.561996) 5 (45.5092,-73.5682)
8 (45.4383,-73.8205) 4 (45.4977,-73.714)
```

```
Truck position: (45.4383,-73.8205)
Distance:0      Number of boxes:0      Position:(45.4383,-73.8205)
Distance:10611.3 Number of boxes:0      Position:(45.4977,-73.714)
Distance:21193.7 Number of boxes:2      Position:(45.5092,-73.5682)
```

STRUCTURE DU PROGRAMME

Le programme Tp1 doit pouvoir être lancé en ligne de commande avec deux arguments obligatoires en entrée : un nom de fichier de données initiales, un nom de fichier résultat, ainsi qu'un argument optionnel n, compris entre 1 et 3, permettant de sélectionner l'algorithme de tri :

- 1 = l'algo simple
- 2 = l'algo efficace
- 3 = l'algo de la librairie standard

Si n n'est pas fourni au moment de l'exécution, vous devez utiliser pour la remise la version la plus efficace de l'algorithme de tri selon vous.

```
java Tp1 nomfichier1.txt nomfichier2.txt n(optionnel)
```

Le fichier `nomfichier2.txt` contient un nombre de boîtes à charger, la capacité du camion et un état d'un entrepôt, soit une liste des bâtiments de cargaison (coordonnées) avec le nombre de boîtes disponibles. Si `nomfichier1.txt` est passé en argument (`args[0]`), alors votre programme doit lire depuis le fichier `nomfichier1.txt`, faire le traitement et enregistrer le résultat dans le format spécifié dans le deuxième fichier `nomfichier2.txt` passé au programme comme deuxième argument (`args[1]`). Considérons le fichier d'entrée `camion_entrepot.txt`. Le fichier `camion_entrepot.txt` est

structuré de la façon suivante. Sur la première ligne, on y retrouve un nombre de boîtes à transporter et la capacité du camion. Les autres lignes contiennent une liste des 2 champs:

- nombre de boîtes au point de service;
- point de service spécifié par les coordonnées (latitude, longitude);

Pour faciliter la lecture (le *parsing*) au moyen d'un flux de lecture des tabulations ou des espaces blancs sépareront les champs. À titre d'exemple, voici un extrait du fichier d'entrée `camion_entrepot.txt` fourni.

```
15 15
2 (45.515399, -73.561996) 5 (45.5092,-73.5682)
8 (45.4383,-73.8205) 4 (45.4977,-73.714)
```

Sortie

La recommandation de cargaison calculée par le programme doit être écrite dans un fichier avec le nom fourni comme deuxième argument au programme.

Truck position: (45.4383,-73.8205)		
Distance:0	Number of boxes:0	Position:(45.4383,-73.8205)
Distance:10611.3	Number of boxes:0	Position:(45.4977,-73.714)
Distance:21193.7	Number of boxes:2	Position:(45.5092,-73.5682)

Le format de sortie: première ligne : “Truck position:”, espace, suivi par un point de service représenté par les coordonnées; Sur chaque ligne suivante : “Distance:” suivi par la valeur de la distance calculée, un espace suivi par la chaîne “Number of boxes:”, la valeur des boîtes restantes à chaque point de service impliqué, un espace, la chaîne “Position:” suivi par les coordonnées. Les espaces pourraient être remplacés par le symbole tabulation. Il faut afficher les points de services impliqués dans la cargaison. Il est très important de respecter ce format de sortie, car des scripts de correction seront utilisés. L'exécution de la commande :

```
java Tp1 camion_entrepot.txt res+.txt
```

doit produire le résultat (`res+.txt`) :

Truck position: (45.4383,-73.8205)		
Distance:0	Number of boxes:0	Position:(45.4383,-73.8205)
Distance:10611.3	Number of boxes:0	Position:(45.4977,-73.714)
Distance:21193.7	Number of boxes:2	Position:(45.5092,-73.5682)

ANALYSE TEMPORELLE

Vous devez réaliser deux types d'analyses de la complexité temporelle des algorithmes pour les 2 versions de votre solution utilisant les différents algorithmes de tri : une analyse expérimentale (empirique) et une analyse théorique (asymptotique), ainsi qu'une analyse empirique de la version utilisant la méthode de tri prédéfinie. Les résultats de ces analyses doivent être inclus dans le rapport.

REMISE

Vous devez remettre électroniquement le TP1 **au plus tard le 15 juin 2026 à 23h59**.

Remise électronique

Vous devez remettre un seul fichier archivé (.zip) contenant tous vos fichiers sources via StudiUM. Le rapport avec l'analyse de la complexité temporelle, incluant les analyses empiriques des 3 versions de votre solution, ainsi que deux analyses asymptotiques des deux versions avec les algorithmes de tri implémentés, peut être remis en version électronique (PDF). L'analyse asymptotique peut être effectuée directement sur le code en annotant les complexités des différents morceaux de code, permettant ainsi de justifier les expressions de la complexité globale de vos deux solutions.

RAPPORT

Le rapport est constitué du/d'un(e)/de:

1. **Auto-évaluation** indiquant si votre programme fonctionne correctement, partiellement ou aucunement.
 2. **Analyse de la complexité temporelle (pire cas) théorique en notation grand O**
 - La complexité temporelle doit être exprimée en fonction de la taille du problème :
 - n indique nombre de bâtiments impliqués dans la cargaison; L'analyse asymptotique doit être faite sur un pseudo code de votre solution ou directement sur le code Java développé.
 3. **Analyse empirique de la complexité temporelle**
 - Les graphiques doivent montrer les temps d'exécution de vos 3 solutions avec différentes valeurs de n. Il est fortement conseillé de créer vos propres fichiers de tests pour les problèmes de différentes tailles afin d'avoir une analyse empirique plus représentative.
-

ÉVALUATION

Ce travail pratique vaut 10% de la note finale.

Grille de correction

Critère	Description	Pondération
A.	Respect des directives pour la remise	/ 0.5
B.	Appréciation générale Structure du programme + Qualité du code : découpage du programme, choix des types de données; identificateurs (noms) significatifs, lisibilité du code, pertinence des commentaires; etc. Encapsulation : respect des principes de l'abstraction;	/ 1

C.	Fonctionnement correct. Le programme produit les bonnes recommandations. Une recommandation non optimale est considérée mauvaise même si elle est très proche de l'optimale. L'efficacité n'est pas directement évaluée. Cependant, l'efficacité peut être indirectement évaluée lorsqu'un programme ne parvient pas à produire des résultats dans des délais raisonnables.	/ 3
D.	Exactitude de l'auto-évaluation. Vous déclarez que votre programme fonctionne correctement ou partiellement ou aucunement.	/ 0.5
E.	Analyse de l'algorithme : théorique et empirique	/ 5
	Total:	10/ 10

Pour les cas problématiques, jusqu'à 2 points peuvent être retranchés pour la qualité de la présentation.

Quelques clarifications :

1. Vous devez placer le camion initialement au bâtiment où il y a le plus de boîtes. Le camion ne bougera plus ensuite. Les monte-charges ne feront que des trajets directs entre le camion et les autres bâtiments (et donc, pas de trucs du genre aller au bâtiment x, puis au bâtiment y, puis aller au camion). On cherche donc à minimiser la distance maximale entre le point où on place le camion et les points où on décide d'aller chercher des boîtes.
2. Il se pourrait que le nombre de boîtes qu'on veut charger dans le camion ne concorde pas avec les autres données. Voici ce que vous devriez faire :

Si le nombre de boîtes demandé est plus grand que la capacité du camion :

- Sois-vous lancez une exception contenant un message approprié;
- Sois-vous ignorez le nombre de boîtes demandé et vous remplissez le camion à sa pleine capacité.

Si le nombre de boîtes demandé est plus petit que la somme des boîtes disponibles dans les entrepôts :

- Sois-vous lancez une exception contenant un message approprié;

- Sois-vous ignorez le nombre de boîtes demandé et vous mettez toutes les boîtes disponibles dans le camion.
3. Dans votre output, n'affichez que les bâtiments sélectionnés (incluant celui où on place initialement le camion), en ordre croissant de distance au camion. Vous ne devriez pas afficher de bâtiments où les monte-charges n'iront pas chercher de boîtes.
 4. Le contenu de votre fichier d'output devrait être dans un format identique à l'exemple dans l'énoncé (même nombre de décimales, même espaces/tabulations aux mêmes endroits, même mots avec les mêmes majuscules/minuscules, etc.)
 5. Exemple de la solution simpliste attendue pour ce TP : s'il vous reste 5 boîtes à charger et que vous avez les options suivantes :
 - 5 bâtiments contenant chacun 1 boîte, à distance 100 du camion
 - 1 bâtiment contenant 5 boîtes, à distance 200 du camion.

Alors la solution attendue serait d'aller chercher chacune des boîtes parmi les 5 bâtiments à distance 100.